

Introduction to the Delta Lake Lakehouse Format

This chapter explains Delta Lake’s origins and how it was initially designed to address data integrity issues around petabyte-scale systems. If you are familiar with Delta Lake’s history and instead want to dive into what Delta Lake is, its anatomy, and the Delta transaction protocol, feel free to jump ahead to the section “[What Is Delta Lake?](#)” on [page 6](#) later in this chapter.

The Genesis of Delta Lake

In this section, we’ll chart the course of Delta Lake’s short evolutionary history: its genesis and inspiration, and its adoption in the community as a lakehouse format, ensuring the integrity of every enterprise’s most important asset: its data. The Delta Lake lakehouse format was developed to address the limitations of traditional data lakes and data warehouses. It provides [ACID \(atomicity, consistency, isolation, and durability\) transactions](#) and scalable metadata handling and unifies various data analytics tasks, such as batch and streaming workloads, machine learning, and SQL, on a single platform.

Data Warehousing, Data Lakes, and Data Lakehouses

There have been many technological advancements in data systems (high-performance computing [HPC] and object databases, for example); a simplified overview of the advancements in querying and aggregating large amounts of business data systems over the last few decades would cover data warehousing, data lakes, and lakehouses. Overall, these systems address online analytics processing (OLAP) workloads.

Data warehousing

Data warehouses are purpose-built to aggregate and process large amounts of structured data quickly (Figure 1-1). To protect this data, they typically use relational databases to provide ACID transactions, a step that is crucial for ensuring data integrity for business applications.

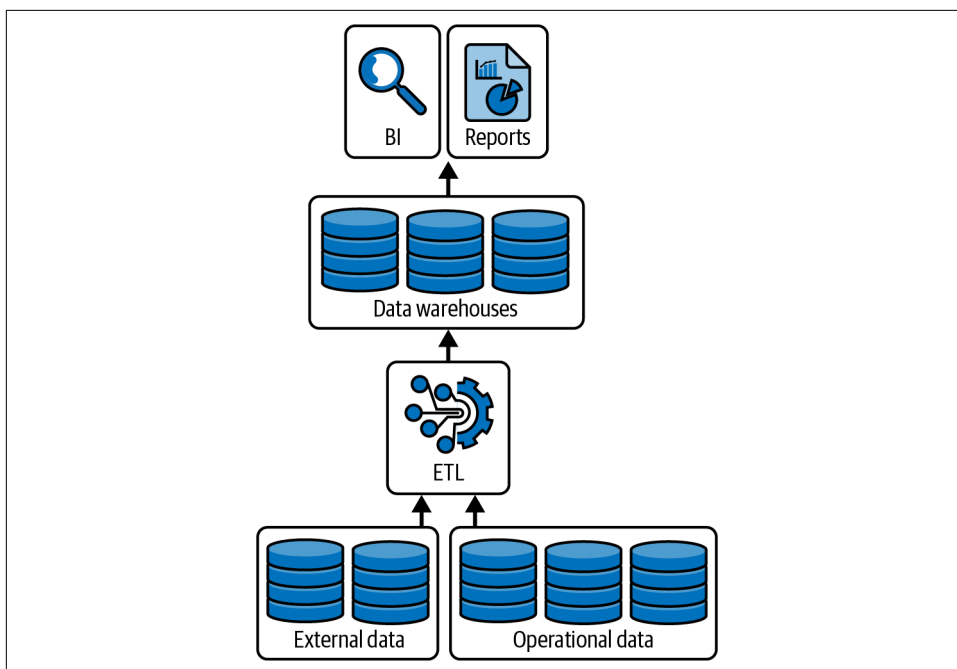


Figure 1-1. Data warehouses are purpose-built for querying and aggregating structured data

Building on the foundation of ACID transactions, data warehouses include management features (backup and recovery controls, gated controls, etc.) to simplify the database operations as well as performance optimizations (indexes, partitioning, etc.) to provide reliable results to the end user more quickly. While robust, data warehouses are often hard to scale to handle the large volumes, variety of analytics (including event processing and data sciences), and data velocity typical in big data scenarios. This limitation is a critical factor that often necessitates using more scalable solutions such as data lakes or distributed processing frameworks like Apache Spark.

Data lakes

Data lakes are scalable storage repositories (HDFS, cloud object stores such as Amazon S3, ADLS Gen2, and GCS, and so on) that hold vast amounts of raw data in their native format until needed (see Figure 1-2). Unlike traditional databases,

data lakes are designed to handle an internet-scale volume, velocity, and variety of data (e.g., structured, semistructured, and unstructured data). These attributes are commonly associated with big data. Data lakes changed how we store and query large amounts of data because they are designed to scale out the workload across multiple machines or nodes. They are file-based systems that work on clusters of commodity hardware. Traditionally, data warehouses were scaled up on a single machine; note that massively parallel processing data warehouses have existed for quite some time but were more expensive and complex to maintain. Also, while data warehouses were designed for structured (or tabular) data, data lakes can hold data in the format of one's choosing, providing developers with flexibility for their data storage.

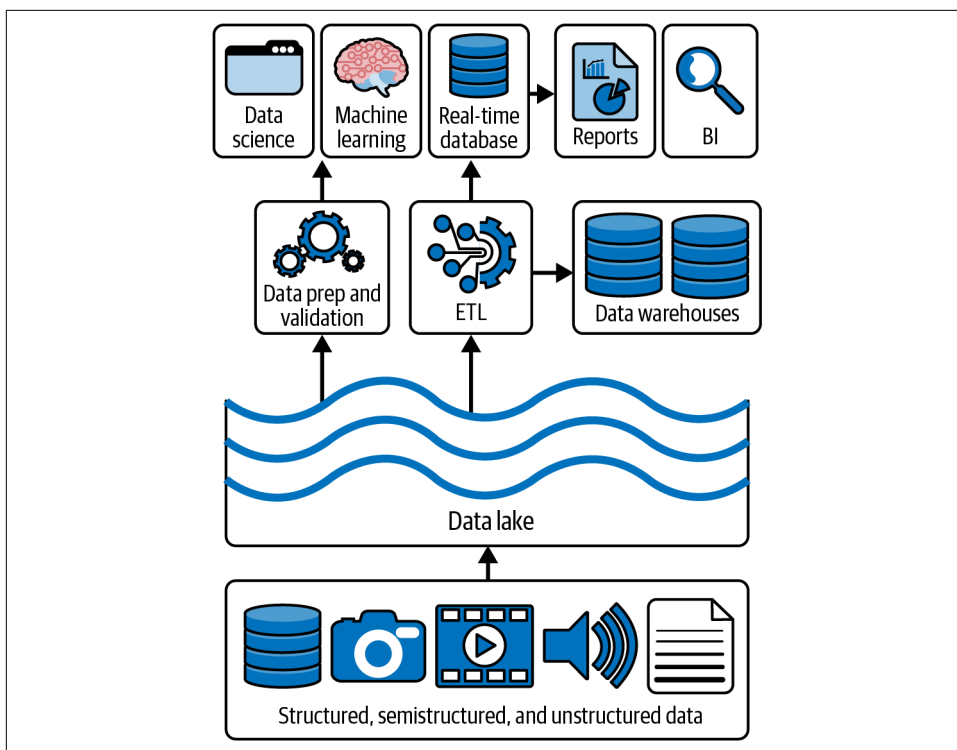


Figure 1-2. Data lakes are built for storing structured, semistructured, and unstructured data on scalable storage infrastructure (e.g., HDFS or cloud object stores)

While data lakes could handle all your data for data science and machine learning, they are an inherently unreliable form of data storage. Instead of providing ACID protections, these systems follow the BASE model—basically available, soft-state, and **eventually consistent**. The lack of ACID guarantees means the storage system processing failures leave your storage in an inconsistent state with orphaned files.

Subsequent queries to the storage system include files that should not result in duplicate counts (i.e., wrong answers).

Together, these shortcomings can lead to an infrastructure poorly suited for BI queries, inconsistent and slow performance, and quite complex setups. Often, the creation of data lakes leads to unreliable data swamps instead of clean data repositories due to the lack of transaction protections, schema management, and so on.

Lakehouses (or data lakehouses)

The lakehouse combines the best elements of data lakes and data warehouses for OLAP workloads. It merges the scalability and flexibility of data lakes with the management features and performance optimization of data warehouses (see [Figure 1-3](#)). There were previous attempts to allow data warehouses and data lakes to coexist side by side. But such an approach was expensive, introducing management complexities, duplication of data, and the reconciliation of reporting/analytics/data science between separate systems. As the practice of data engineering evolved, the concept of the *lakehouse* was born. A lakehouse eliminates the need for disjointed systems and provides a single, coherent platform for all forms of data analysis. Lakehouses enhance the performance of data queries and simplify data management, making it easier for organizations to derive insights from their data.

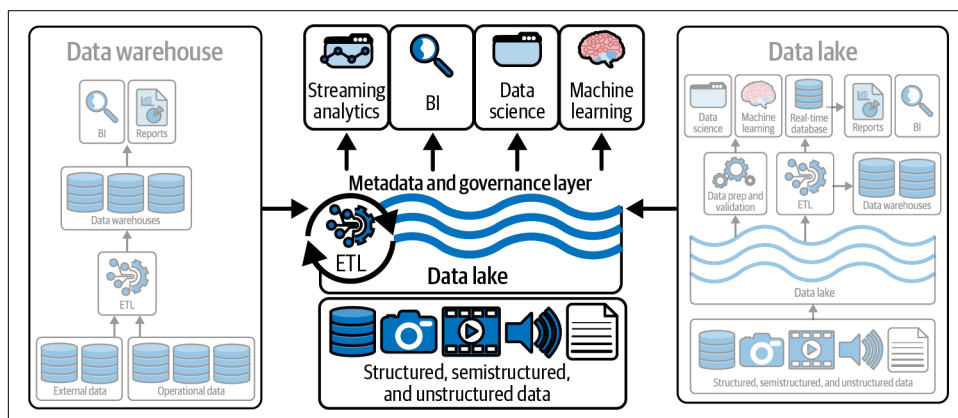


Figure 1-3. Lakehouses are the best of both worlds between data warehouses and data lakes

Delta Lake, Apache Iceberg, and Apache Hudi are the most popular open source lakehouse formats. As you can guess, this book will focus on Delta Lake.¹

¹ To learn more about lakehouses, see the 2021 CIDR whitepaper “[Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics](#)”.

Project Tahoe to Delta Lake: The Early Years Months

The 2021 online meetup [From Tahoe to Delta Lake](#) provided a nostalgic look back at how Delta Lake was created. The panel featured “old school” developers and Delta Lake maintainers Burak Yavuz, Denny Lee, Ryan Zhu, and Tathagata Das, as well as the creator of Delta Lake, Michael Armbrust. It also included the “new school” Delta Lake maintainers who created the delta-rs project, QP Hou and R. Tyler Croy.

The original project name for Delta Lake was “Project Tahoe,” as Michael Armbrust had the initial idea of providing transactional reliability for data lakes while skiing at Tahoe in 2017. Lake Tahoe is an iconic and massive lake in California, symbolizing the large-scale data lake the project aimed to create. Michael is a committer/PMC member of Apache Spark™; a Delta Lake maintainer; one of the original creators of Spark SQL, Structured Streaming, and Delta Lake; and a distinguished software engineer at Databricks. The transition from “Tahoe” to “Delta Lake” occurred around New Year’s 2018 and came from Jules Damji. The rationale behind changing the name was to invoke the natural process in which rivers flow into deltas, depositing sediments that eventually build up and create fertile ground for crops. This metaphor was fitting for the project, as it represented the convergence of data streams into a managed data lake, where data practitioners could cultivate valuable insights. The Delta name also resonated with the project’s architecture, which was designed to handle massive and high-velocity data streams, allowing the data to be processed and split into different streams or views.

But why did Armbrust create Delta Lake? He created it to address the limitations of Apache Spark’s file synchronization. Specifically, he wanted to handle large-scale data operations and needed robust transactional support. Thus, his motivation for developing Delta Lake stemmed from the need for a scalable transaction log that could handle massive data volumes and complex operations.

Early in the creation of Delta Lake are two notable use cases that emphasize its efficiency and scalability. Comcast utilized Delta Lake to enhance its data analytics and machine learning platforms and manage its petabytes of data. This transition reduced its compute utilization from 640VMs to 64VMs and simplified job maintenance from 84 to 3 jobs. By streamlining its processing with Delta Lake, Comcast reduced its compute utilization by 10x, with 28x fewer jobs. Apple’s information security team employed Delta Lake for real-time threat detection and response, handling over 300 billion events per day and writing hundreds of terabytes of data daily. Both cases illustrate Delta Lake’s superior performance and cost-effectiveness compared to traditional data management methods. We will look at additional use cases in [Chapter 11](#).

What Is Delta Lake?

Delta Lake is an open source storage layer that supports ACID transactions, scalable metadata handling, and unification of streaming and batch data processing. It was initially designed to work with Apache Spark and large-scale data lake workloads.

With Delta Lake, you can build a single data platform with your choice of high-performance query engine to address a diverse range of workloads, including (but not limited to) business intelligence (BI), streaming analytics/complex event processing, data science, and machine learning, as noted in **Figure 1-4**.

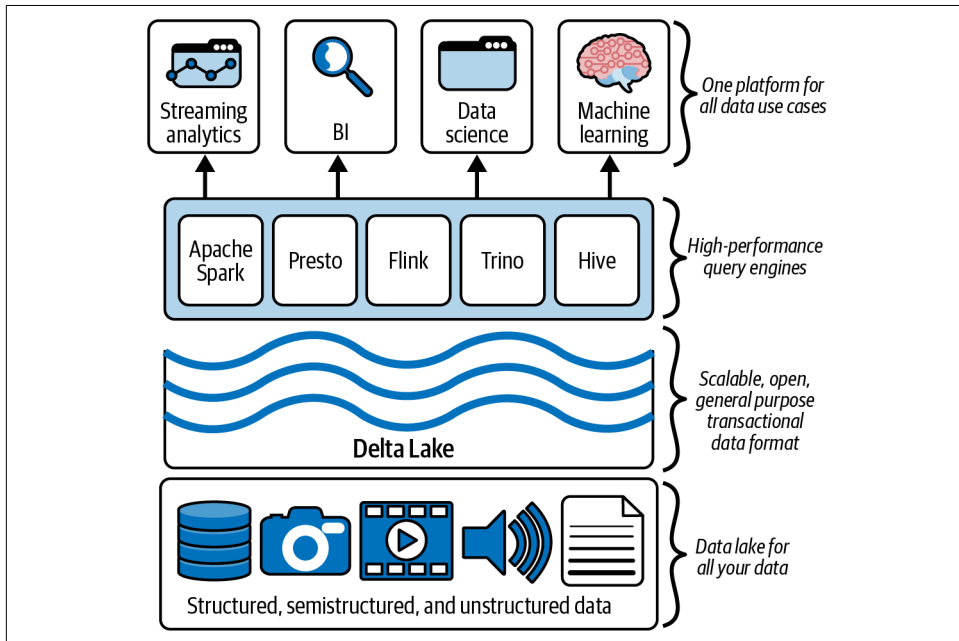


Figure 1-4. Delta Lake provides a scalable, open, general-purpose transactional data format for your lakehouse

However, as it has evolved, Delta Lake has been optimally designed to work with numerous workloads (small data, medium data, big data, etc.). It has also been designed to work with multiple frameworks (e.g., Apache Spark, Apache Flink, Trino, Presto, Apache Hive, and Apache Druid), services (e.g., Athena, Big Query, Databricks, EMR, Fabric, Glue, Starburst, and Snowflake), and languages (.NET, Java, Python, Rust, Scala, SQL, etc.).

Common Use Cases

Developers in all types of organizations, from startups to large enterprises, use Delta Lake to manage their big data and AI workloads. Common use cases include:

Modernizing data lakes

Delta Lake helps organizations modernize their data lakes by providing ACID transactions, scalable metadata handling, and schema enforcement, thereby ensuring data reliability and performance improvements.

Data warehousing

There are both data warehousing technologies and *techniques*. The Delta Lake lakehouse format allows you to apply data warehousing techniques to provide fast query performance for various analytics workloads while also providing data reliability.

Machine learning/data science

Delta Lake provides a reliable data foundation for machine learning and data science teams to access and process data, enabling them to build and deploy models faster.

Streaming data processing

Delta Lake unifies streaming and batch data processing. This allows developers to process real-time data and perform complex transformations on the fly.

Data engineering

Delta Lake provides a reliable and performant platform for data engineering teams to build and manage data pipelines, ensuring data quality and accuracy.

Business intelligence

Delta Lake supports SQL queries, making it easy for business users to access and analyze data and thus enabling them to make data-driven decisions.

Overall, Delta Lake is used by various teams, including data engineers, data scientists, and business users, to manage and analyze big data and AI workloads, ensuring data reliability, performance, and scalability.

Key Features

Delta Lake comprises the following key features that are fundamental to an open lakehouse format (please see the VLDB research article “[Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores](#)” for a deeper dive into these features):

ACID transactions

Delta Lake ensures that data modifications are performed atomically, consistently, in isolation, and durably, i.e., with ACID transaction protections. This means that when multiple concurrent clients or tasks access the data, the system maintains data integrity. For instance, if a process fails during a data modification, Delta Lake will roll back the changes, ensuring that the data remains consistent.

Scalable metadata

The metadata of a Delta Lake table is the transaction log, which provides transactional consistency per the aforementioned ACID transactions. With a petabyte-scale table, the table’s metadata can itself be exceedingly complicated to maintain. Delta Lake’s scalable metadata handling feature is designed to manage metadata efficiently for large-scale datasets without its operations impacting query or processing performance.

Time travel

The Delta Lake time travel feature allows you to query previous versions of a table to access historical data. Made possible by the Delta transaction log, it enables you to specify a version or timestamp to query a specific version of the data. This is very useful for a variety of use cases, such as data audits, regulatory compliance, and data recovery.

Unified batch/streaming

Delta Lake was designed hand in hand with Apache Spark Structured Streaming to simplify the logic around streaming. Instead of having different APIs for batch and streaming, Structured Streaming uses the same in-memory Datasets/Data-Frame API for both scenarios. This allows developers to use the same business logic and APIs, the only difference being latency. Delta Lake provides the ACID guarantees of the storage system to support this unification.

Schema evolution/enforcement

Delta Lake’s schema evolution and schema enforcement ensure data consistency and quality by enforcing a schema on write operations and allowing users to modify the schema without breaking existing queries. They also prevent developers from inadvertently inserting data with incorrect columns or types, which is crucial for maintaining data quality and consistency.

Audit history

This feature provides detailed logs of all changes made to the data, including information about who made each change, what the change was, and when it was made. This is crucial for compliance and regulatory requirements, as it allows users to track changes to the data over time and ensure that data modifications are performed correctly. The Delta transaction log makes all of this possible.

DML operations

Delta Lake was one of the first lakehouse formats to provide data manipulation language (DML) operations. This initially extended Apache Spark to support various operations such as insert, update, delete, and merge (or CRUD operations). Today, users can effectively modify the data using multiple frameworks, services, and languages.

Open source

The roots of Delta Lake were built within the foundation of Databricks, which has extensive experience in open source (the founders of Databricks were the original creators of Apache Spark). Shortly after its inception, Delta Lake was donated to the Linux Foundation to ensure developers have the ability to use, modify, and distribute the software freely while also promoting collaboration and innovation within the data engineering community.

Performance

While Delta Lake is a lakehouse storage format, it is optimally designed to improve the speed of your queries and processing for both ingestion and querying using the default configuration. While you can continually tweak the performance of Delta Lake, most of the time the defaults will work for your scenarios.

Ease of use

Delta Lake was built with simplicity in mind right from the beginning. For example, to write a table using Apache Spark in Parquet file format, you would execute:

```
data.write.format("parquet").save("/tmp/parquet-table")
```

To do the same thing for Delta, you would execute:

```
data.write.format("delta").save("/tmp/delta-table")
```

Anatomy of a Delta Lake Table

A Delta Lake table or Delta table comprises several key components that work together to provide a robust, scalable, and efficient data storage solution. The main elements are as follows:

Data files

Delta Lake tables store data in Parquet file format. These files contain the actual data and are stored in a distributed cloud or on-premises file storage system such as HDFS (Hadoop Distributed File System), Amazon S3, Azure Blob Storage (or Azure Data Lake Storage [ADLS] Gen2), GCS (Google Cloud Storage), or MinIO. Parquet was chosen for its efficiency in storing and querying large datasets.

Transaction log

The transaction log, also known as the Delta log, is a critical component of Delta Lake. It is an ordered record of every transaction performed on a Delta Lake table. The transaction log ensures ACID properties by recording all changes to the table in a series of JSON files. Each transaction is recorded as a new JSON file in the `_delta_log` directory, which includes metadata about the transaction, such as the operation performed, the files added or removed, and the schema of the table at the time of the transaction.

Metadata

Metadata in Delta Lake includes information about the table's schema, partitioning, and configuration settings. This metadata is stored in the transaction log and can be retrieved using SQL, Spark, Rust, and Python APIs. The metadata helps manage and optimize the table by providing information for schema enforcement and evolution, partitioning strategies, and data skipping.

Schema

A Delta Lake table's schema defines the data's structure, including its columns, data types, and so on. The schema is enforced on write, ensuring that all data written to the table adheres to the defined structure. Delta Lake supports schema evolution (add new columns, rename columns, etc.), allowing the schema to be updated as the data changes over time.

Checkpoints

Checkpoints are periodic snapshots of the transaction log that help speed up the recovery process. Delta Lake consolidates the state of the transaction log by default every 10 transactions. This allows client readers to quickly catch up from the most recent checkpoint rather than replaying the entire transaction log from the beginning. Checkpoints are stored as Parquet files and are created automatically by Delta Lake.

Figure 1-5 is a graphical representation of the structure of a Delta Lake table.

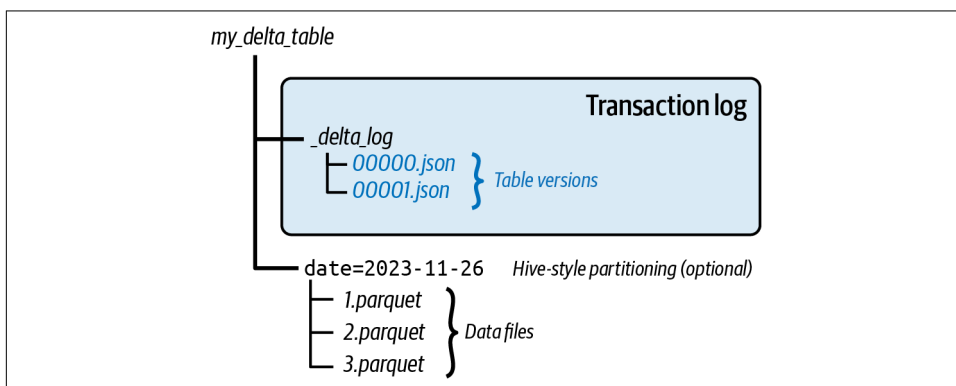


Figure 1-5. Delta Lake table layout for the transaction log and data files (adapted from an image by Denny Lee)²

Delta Transaction Protocol

In the previous section, we described the anatomy of a Delta Lake table. The **Delta transaction log protocol** is the specification defining how clients interact with the table in a consistent manner. At its core, all interactions with the Delta table must begin by reading the Delta transaction log to know what files to read. When a client modifies the data, the client initiates the creation of new data files (i.e., Parquet files) and then inserts new metadata into the transaction log to commit modifications to the table. In fact, many of the original **Delta Lake integrations** (delta-spark, Trino connector, delta-rust API, etc.) had codebases maintained by different communities. A Rust client could write, a Spark client could modify, and a Trino client could read from the same Delta table without conflict because they all independently followed the same protocol.

Implementing this specification brings ACID properties to large data collections stored as files in a distributed filesystem or object store. As defined in the specification, the protocol was designed with the following goals in mind:

Serializable ACID writes

Multiple writers can modify a Delta table concurrently while maintaining ACID semantics.

² Denny Lee, “[Understanding the Delta Lake Transaction Log at the File Level](#)”, Denny Lee (blog), November 26, 2023.

Snapshot isolation for reads

Readers can read a consistent snapshot of a Delta table, even in the face of concurrent writes.

Scalability to billions of partitions or files

Queries against a Delta table can be planned on a single machine or in parallel.

Self-describing

All metadata for a Delta table is stored alongside the data. This design eliminates the need to maintain a separate metastore to read the data and allows static tables to be copied or moved using standard filesystem tools.

Support for incremental processing

Readers can tail the Delta log to determine what data has been added in a given period of time, allowing for efficient streaming.

Understanding the Delta Lake Transaction Log at the File Level

To better understand this in action, let's look at what happens at the file level when a Delta table is created. Initially, the table's transaction log is automatically created in the `_delta_log` subdirectory. As changes are made to the table, the operations are recorded as ordered *atomic commits* in the transaction log. Each commit is written out as a JSON file, starting with `000...00000.json`. Additional changes to the table generate subsequent JSON files in ascending numerical order, so that the next commits are written out as `000...00001.json`, `000...00002.json`, and so on. Each numeric JSON file increment represents a new version of the table, as described in [Figure 1-5](#).

Note how the structure of the data files has not changed; they exist as separate Parquet files generated by the query engine or language writing to the Delta table. If your table utilizes Hive-style partitioning, you will retain the same structure.

The Single Source of Truth

Delta Lake allows multiple readers and writers of a given table to all work on the table at the same time. It is the central repository that tracks all user changes to the table. This concept is important because, over time, processing jobs will invariably fail in your data lake. The result is partial files that are not removed. Subsequent processing or queries will not be able to ascertain which files should or should not be included in their queries. To show users correct views of the data at all times, the Delta log is the *single source of truth*.

The Relationship Between Metadata and Data

As the Delta transaction log is the single source of truth, any client who wants to read or write to your Delta table *must* first query the transaction log. For example, when inserting data while creating our Delta table, we initially generate two Parquet files: *1.parquet* and *2.parquet*. This event would automatically be added to the transaction log and saved to disk as *commit 000...00000.json* (see A in [Figure 1-6](#)).

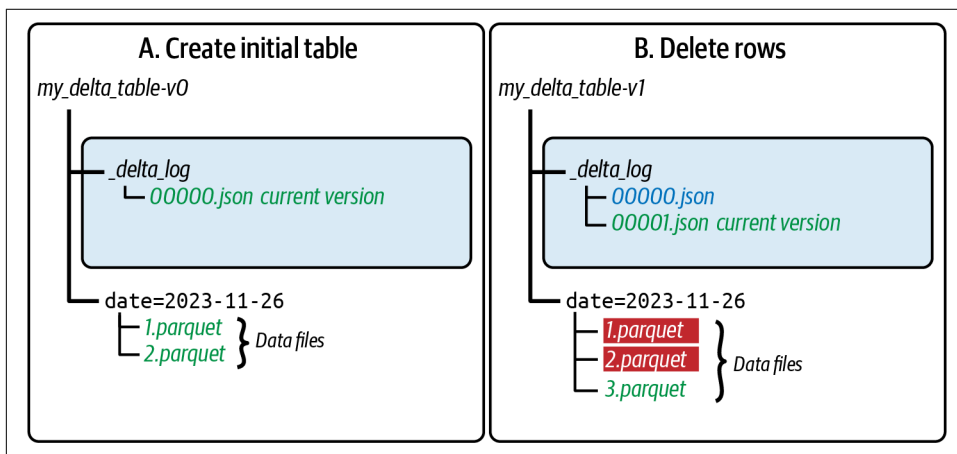


Figure 1-6. (left) Creating a new Delta table by adding Parquet files and their relationship with the Delta transaction log; (right) deleting rows from this Delta table by removing and adding files and their relationship with the Delta transaction log

In a subsequent command (B in [Figure 1-6](#)), we run a DELETE operation that results in the removal of rows from the table. Instead of modifying the existing Parquet files (*1.parquet*, *2.parquet*), Delta creates a third file (*3.parquet*).

Multiversion Concurrency Control (MVCC) File and Data Observations

For deletes on object stores, it is faster to create a new file or files comprising the unaffected rows rather than modifying the existing Parquet file(s). This approach also provides the advantage of multiversion concurrency control (MVCC). MVCC is a database optimization technique that creates copies of the data, thus allowing data to be safely read and updated concurrently. This technique also allows Delta Lake to provide time travel. Therefore, Delta Lake creates multiple files for these actions, providing atomicity, MVCC, and speed.



We can speed up this process by using deletion vectors, an approach we will describe in [Chapter 8](#).

The removal/creation of the Parquet files shown in B in [Figure 1-6](#) is wrapped in a single transaction recorded in the Delta transaction log in the file `000...00001.json`. Some important observations concerning atomicity are:

- If a user were to read the Parquet files without reading the Delta transaction log, they would read duplicates because of the replicated rows in all the files (`1.parquet`, `2.parquet`, `3.parquet`).
- The remove and add actions are wrapped in the single transaction log `000...00001.json`. When a client queries the Delta table at this time, it records both of these actions and the filepaths for that snapshot. For this transaction, the filepath would point only to `3.parquet`.
- Note that the remove operation is a soft delete or tombstone where the physical removal of the files (`1.parquet`, `2.parquet`) has yet to happen. The physical removal of files will happen when executing the `VACUUM` command.
- The previous transaction `000...00000.json` has the filepath pointing to the original files (`1.parquet`, `2.parquet`). Thus, when querying for an older version of the Delta table via time travel, the transaction log points to the files that make up that older snapshot.

Observing the Interaction Between the Metadata and Data

While we now have a better understanding of what happens at the *individual* data file and metadata file level, how does this all work together? Let's look at this problem by following the flow of [Figure 1-7](#), which represents a common data processing failure scenario. The table is initially represented by two Parquet files (`1.parquet` and `2.parquet`) at t_0 .

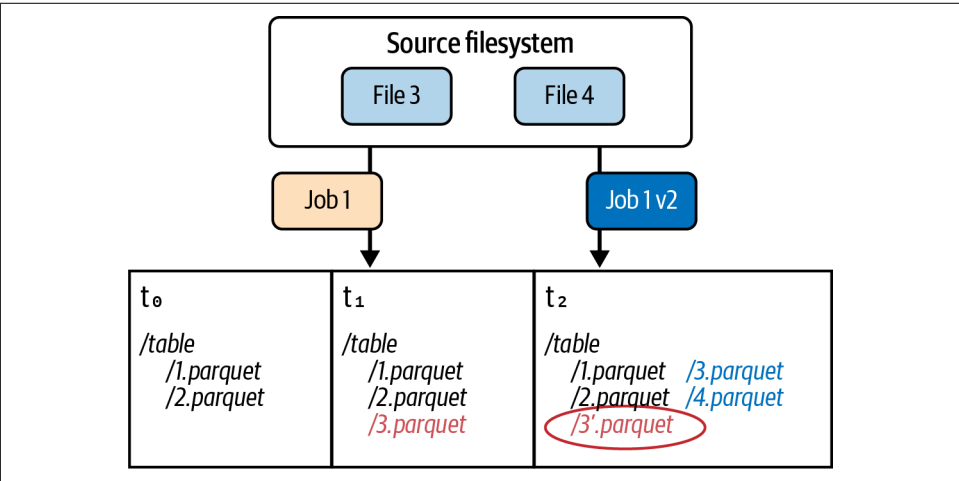


Figure 1-7. A common data processing failure scenario: partial files

At t_1 , job 1 extracts file 3 and file 4 and writes them to storage. However, due to some error (network hiccup, storage temporarily offline, etc.), an incomplete portion of file 3 and none of file 4 are written into *3.parquet*. Thus, *3.parquet* is a partial file, and this incomplete data will be returned to any clients that subsequently query the files that make up this table.

To complicate matters, at t_2 , a new version of the same processing job (job 1 v2) successfully completes its task. It generates a new version of *3.parquet* and *4.parquet*. But because the partial *3'.parquet* (circled) exists alongside *3.parquet*, any system querying these files will result in double counting.

However, because the Delta transaction log tracks which files are valid, we can avoid the preceding scenario. Thus, when a client reads a Delta Lake table, the engine (or API) initially verifies the transaction log to see what new transactions have been posted to the table. It then updates the client table with any new changes. This ensures that any client's version of a table is always synchronized. Clients cannot make divergent, conflicting changes to a table.

Let's repeat the same partial file example on a Delta Lake table. **Figure 1-8** shows the same scenario in which the table is represented by two Parquet files (i.e., *1.parquet* and *2.parquet*) at t_0 . The transaction log records that these two files make up the Delta table at t_0 (Version 0).

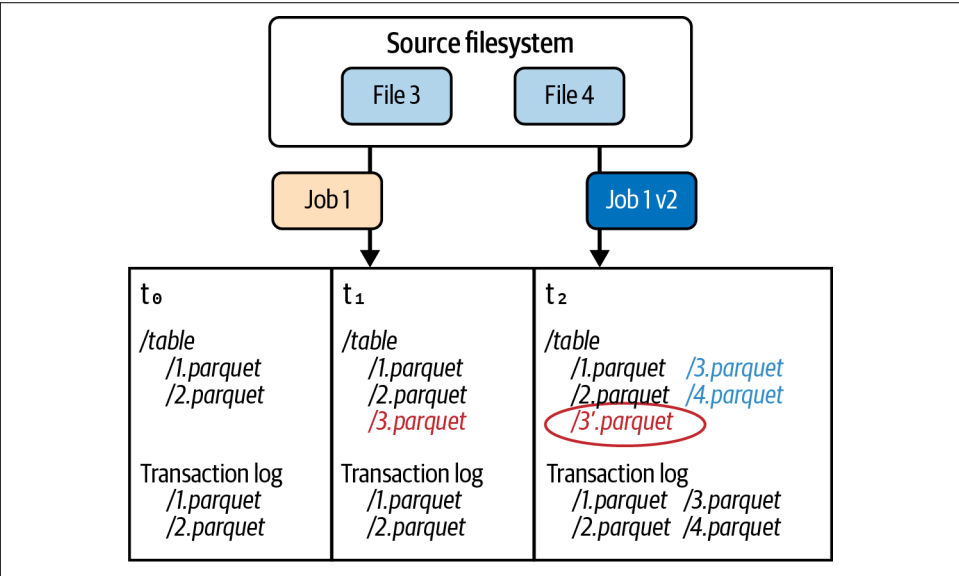


Figure 1-8. Delta Lake avoids the partial files scenario because of its transaction log

At t_1 , job 1 fails with the creation of *3.parquet*. However, because the job failed, the transaction was *not* committed to the transaction log. No new files are recorded;

notice how the transaction log has only *1.parquet* and *2.parquet* listed. Any queries against the Delta table at t_1 will read only these two files, even if other files are in storage.

At t_2 , job 1 v2 is completed, and its output is the files *3.parquet* and *4.parquet*. Because the job was successful, the Delta log includes entries only for the two successful files. That is, *3'.parquet* is *not* included in the log. Therefore, any clients querying the Delta table at t_2 will see only the correct files.

Table Features

Originally, Delta tables used **protocol versions** to map to a set of features to ensure user workloads did not break when new features in Delta were released. For example, if a client wanted to use Delta's **Change Data Feed (CDF) option**, users were required to upgrade their protocol versions and validate their workloads to access new features (Figure 1-9). This ensured that any readers or writers incompatible with a specific protocol version were blocked from reading or writing to that table to prevent data corruption.

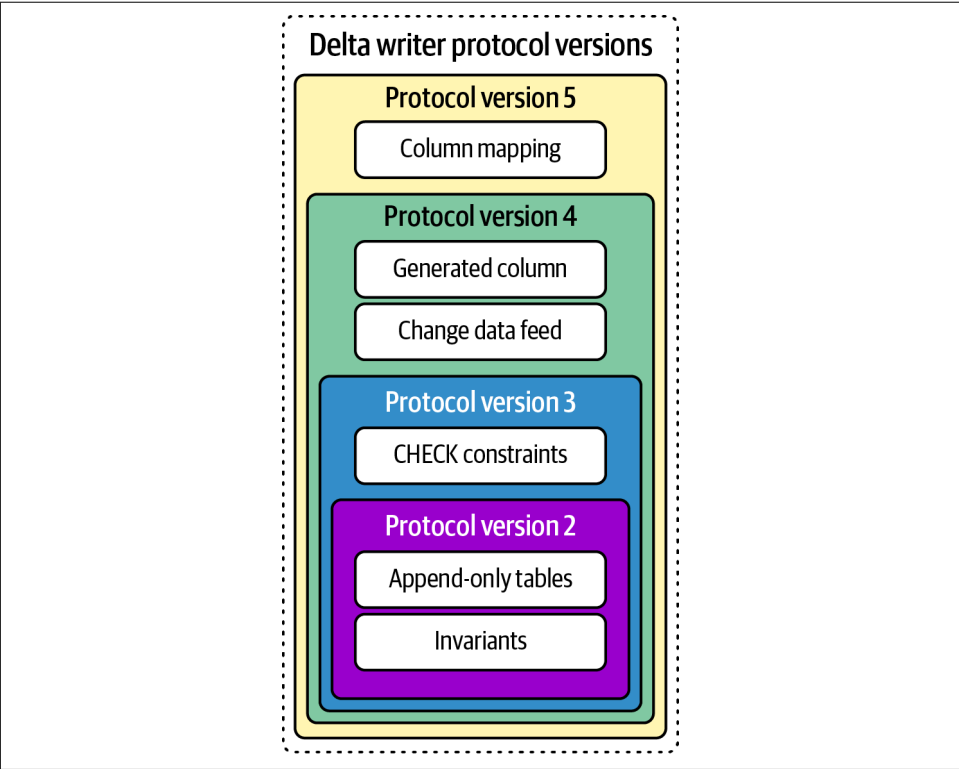


Figure 1-9. Delta writer protocol versions

But this process slows feature adoption because it requires the client and table to support *all* features in that protocol version. For example, with protocol version 4, your Delta table supports both **generated columns** and CDF. For your client to read this table, it must support both generated columns and Change Data Feed even if you only want to use CDF. In other words, Delta connectors have no choice but to implement all features just to support a single feature in the new version.

Introduced in **Delta Lake 2.3.0**, *Table Features* replaces table protocol versions to represent features a table uses so connectors can know which features are required to read or write a table (**Figure 1-10**).

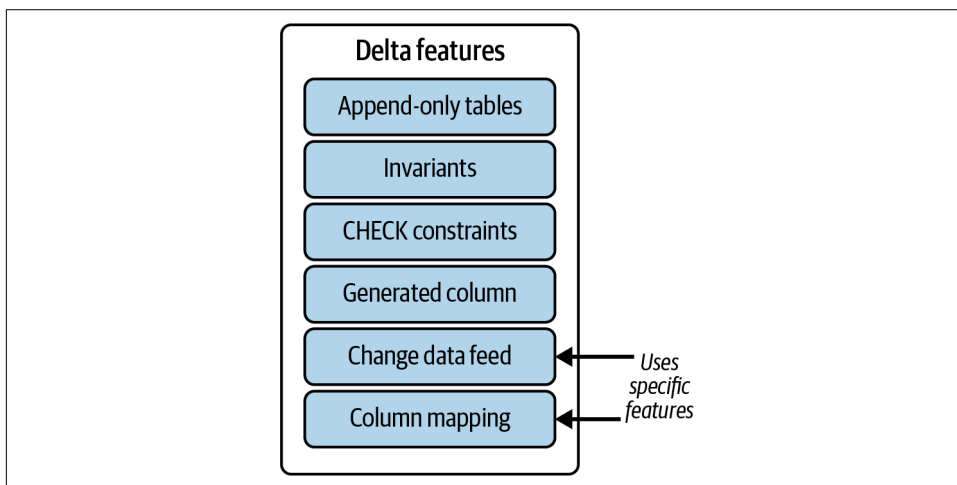


Figure 1-10. Delta Lake Table Features

The advantage of this approach is that any connectors (or integrations) can selectively implement certain features of their interest, instead of having to work on all of them. A quick way to view what table features are enabled is to run the query `SHOW TBLPROPERTIES`:

```
SHOW TBLPROPERTIES default.my_table;
```

The output would look similar to the following:

Key (String)	Value (String)
delta.minReaderVersion	3
delta.minWriterVersion	7
delta.feature.deletionVectors	supported
delta.enableDeletionVectors	true
delta.checkpoint.writeStatsAsStruct	true
delta.checkpoint.writeStatsAsJson	false

To dive deeper, please refer to “[Table Features](#)” in the [GitHub page](#) for the Delta transaction protocol.

Delta Kernel

As previously noted, Delta Lake provides ACID guarantees and performance across many frameworks, services, and languages. As of this writing, every time new features are added to Delta Lake, the connector must be rewritten entirely, because there is a tight coupling between the metadata and data processing. [Delta Kernel](#) simplifies the development of connectors by abstracting out all the protocol details so the connectors do not need to understand them. Kernel itself implements the Delta transaction log specification (per the previous section). This allows the connectors to build only against the Kernel library, which provides the following advantages:

Modularity

Creating Delta Kernel allows for more easily maintained parity between Delta Lake Rust and Scala/JVM, enabling both to be first-class citizens. All metadata (i.e., transaction log) logic is coordinated and executed through the Kernel library. This way, the connectors need only to focus on how to perform their respective frameworks/services/languages. For example, the Apache Flink/Delta Lake connector needs to focus only on reading or modifying the specific files provided by Delta Kernel. The end client does not need to understand the semantics of the transaction log.

Extensibility

Delta Kernel decouples the *logic* for the metadata (i.e., transaction log) from the data. This allows Delta Lake to be modular, extensible, and highly portable (for example, you can copy the entire table with its transaction log to a new location for your AI workloads). This also extends (pun intended) to Delta Lake’s extensibility, as a connector is now, for example, provided the list of files to read instead of needing to query the transaction log directly. Delta Lake already has many [integrations](#), and by decoupling the logic around the metadata from the data, it will be easier for all of us to maintain our various connectors.

Delta Kernel achieves this level of abstraction through the following requirements:

It provides narrow, stable APIs for connectors.

For a table scan query, a connector needs to specify only the query schema, so that the Kernel can read only the required columns, and the query filters for Kernel to skip data (files, rowgroups, etc.). APIs will be stable and backward compatible. Connectors should be able just to upgrade the Delta Kernel version without rewriting their client code—that is, they automatically get support for an updated Delta protocol via Table Features.

It internally implements the protocol-specific logic.

Delta Kernel will implement all of the following operations:

- Read JSON files
- Read Parquet log files
- Replay log with data skipping
- Read Parquet data and DV files
- Transform data (e.g., filter by DVs)

While Kernel internally implements the protocol-specific logic, better engine-specific implementations can be added (e.g., Apache Spark or Trino may have better JSON and Parquet reading capabilities).

It provides APIs for plugging in better performance.

These include *Table APIs* for connectors to perform table operations such as data scans and *Engine APIs* for plugging in connector-optimized implementations for performance-sensitive components.

As of this writing, Delta Kernel is still in the early stages, and building your own Kernel connector is outside the scope of this book. If you would like to dive deeper into how to build your own Kernel connector, please refer to the following resources:

- “[Umbrella Feature Request] Delta Kernel APIs to simplify building connectors for reading Delta tables”
- Delta Kernel—Java
- Delta Kernel—Rust
- “Delta Kernel: Simplifying Building Connectors for Delta”

Delta UniForm

As noted in the section “[Lakehouses \(or data lakehouses\)](#)” on [page 4](#), there are multiple lakehouse formats. Delta Universal Format, or UniForm, is designed to simplify the interoperability among Delta Lake, Apache Iceberg, and Apache Hudi. Fundamentally, lakehouse formats are composed of metadata and data (typically in Parquet file format).

What makes these lakehouse formats different is how they create, manage, and maintain the *metadata* associated with this data. With Delta UniForm, the metadata of other lakehouse formats is generated concurrently with the Delta format. This way, whether you have a Delta, Iceberg, or Hudi client, it can read the data, because all of their APIs can understand the metadata. Delta UniForm includes the following support:

- Apache Iceberg support as part of [Delta Lake 3.0.0](#) (October 2023)
- Apache Hudi support as part of [Delta Lake 3.2.0](#) (May 2024)

For the latest information on how to enable these features, please refer to the [Delta UniForm documentation](#).

Conclusion

In this chapter, we explained the origins of Delta Lake, what it is and what it does, its anatomy, and the transaction protocol. We emphasized that the Delta transaction log is the single source of truth and thus is the single source of the relationship between its metadata and data. While still early, this has led to the development of Delta Kernel as the foundation for simplifying the building of Delta connectors for Delta Lake's many frameworks, services, and community projects. The core difference between the different lakehouse formats is their metadata, so Delta UniForm unifies them by generating all formats' metadata.